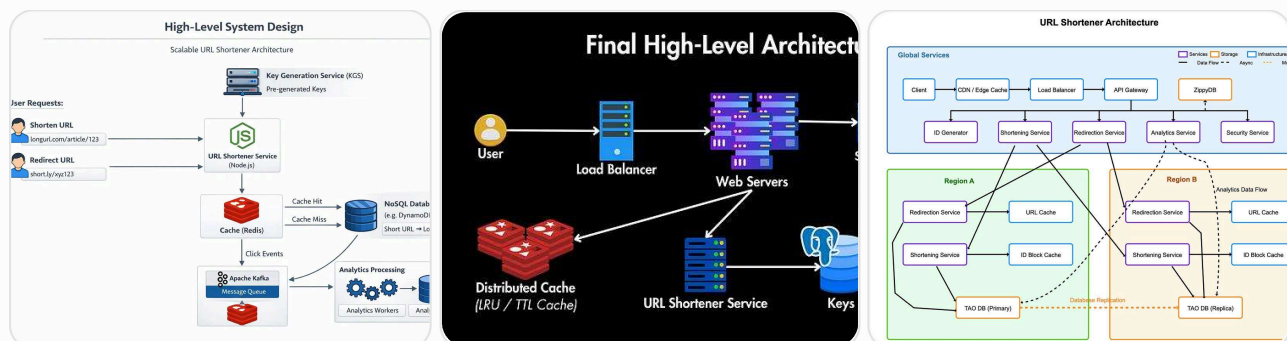


Yesterday 10:55 PM

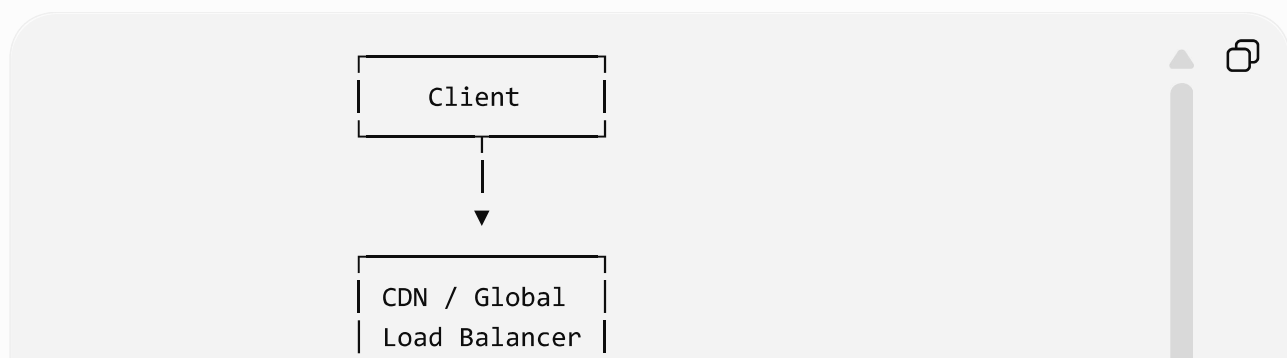
1. How do we generate **unique short IDs**?
2. How do we generate them at **billions of URLs**?
3. How do we avoid collisions?
4. Should we use **Base62**, hashing, UUID, or distributed ID generation?
5. How do we partition/shard the database?
6. How does Redis caching work?
7. How do we handle **millions/billions of redirects**?
8. How do we collect analytics without slowing redirects?
9. How do we handle expired/deleted URLs?
10. How do we make the system highly available across regions?

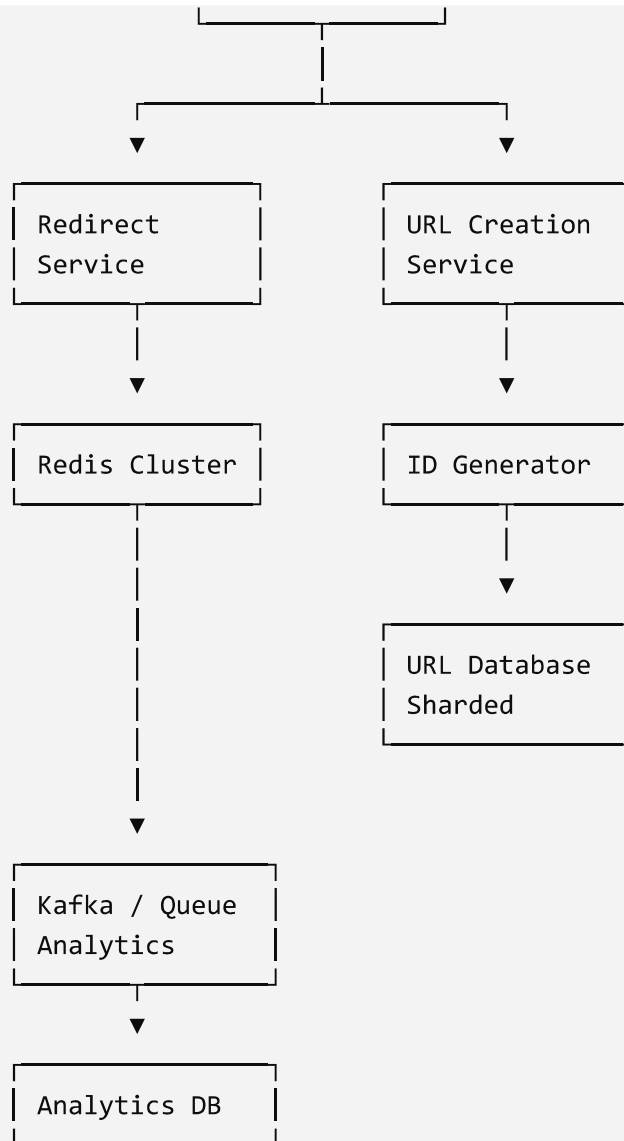


This is the **core architecture problem** of a **production-grade URL Shortener**. The key is to separate **ID generation**, **storage**, **redirect serving**, and **analytics** so the redirect path stays extremely fast.



High-level architecture





1. How do we generate unique short IDs?

The important distinction is:

Base62 is an encoding mechanism, not a uniqueness mechanism.

Suppose we have:

Characters:

0-9 = 10

a-z = 26

A-Z = 26

Total = 62



A 7-character Base62 ID gives:

62^7
= 3,521,614,606,208

≈ 3.5 trillion combinations

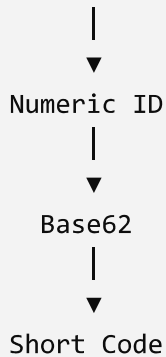


So 7 characters are enough for enormous scale.

But we still need a **unique number** before encoding it.

A good production approach is:

Distributed ID Generator



For example:

Numeric ID:



125678912345



Base62 encoding



"b7Kx92P"

2. How do we generate IDs at billions of URLs?

You don't want every application server asking the database:

```
SELECT MAX(id) + 1
```



That would become a bottleneck.

Instead, use a **distributed ID generator**.

A common design is similar to Snowflake:

64-bit ID



Timestamp	Machine/Node	Sequence
41 bits	10 bits	12 bits

For example:

timestamp

+

machine ID

+

sequence

|

▼

unique 64-bit integer

|

▼

Base62

|

▼

short URL



The sequence allows one machine to generate many IDs within the same millisecond.

3. How do we avoid collisions?

There are several strategies.

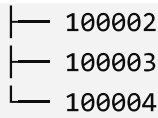
Strategy A — Database auto-increment

DB

|

— 100001





```
├─ 100002
├─ 100003
└─ 100004
```

Very simple, but difficult to scale globally.

Strategy B — Snowflake-style IDs

Each node owns a unique worker ID:

```
Node 1 → worker_id = 1
Node 2 → worker_id = 2
Node 3 → worker_id = 3
```



Combined with timestamp + sequence:

```
timestamp + worker_id + sequence
```



Collision probability is effectively eliminated if worker IDs are correctly assigned.

Strategy C — Database ID + Base62

This is actually an excellent choice for many URL shorteners.

```
Database sequence
```

```
  |
  ▼
123456789
  |
  ▼
Base62
  |
  ▼
"8M0kL"
```



The database guarantees uniqueness.

For a learning/production portfolio system, I'd recommend:

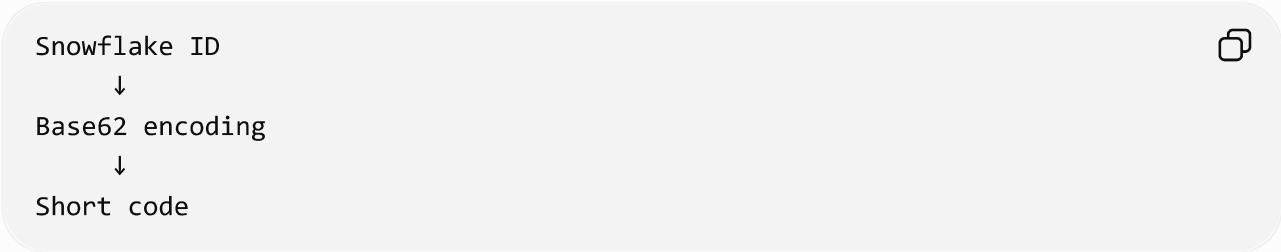
```
Snowflake-style ID
+
Base62 encoding
```



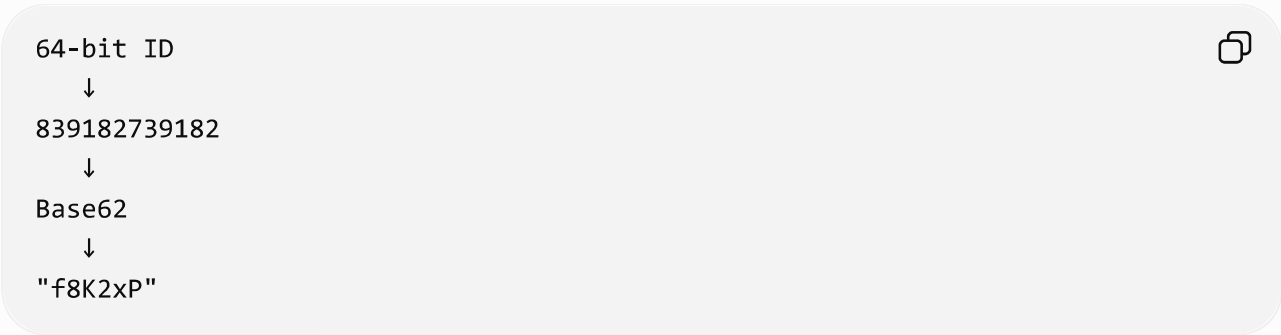
4. Base62 vs Hashing vs UUID vs Distributed IDs

Approach	Short	Collision-free	Distributed	Recommendation
Base62	✓	✗	✗	Encoding only
Hash	✓	✗	✓	Usually not ideal
UUID	✗	✓	✓	Too long for short URLs
DB sequence	✓	✓	⚠	Great for simpler systems
Snowflake ID	✓	✓	✓	★ Excellent
Random Base62	✓	⚠	✓	Collision checking required

Best architecture



Example:



5. How do we partition/shard the database?

Suppose we have:



One database becomes difficult to manage.

We can shard by the short ID.

For example:

```
hash(shortCode) % 16
```

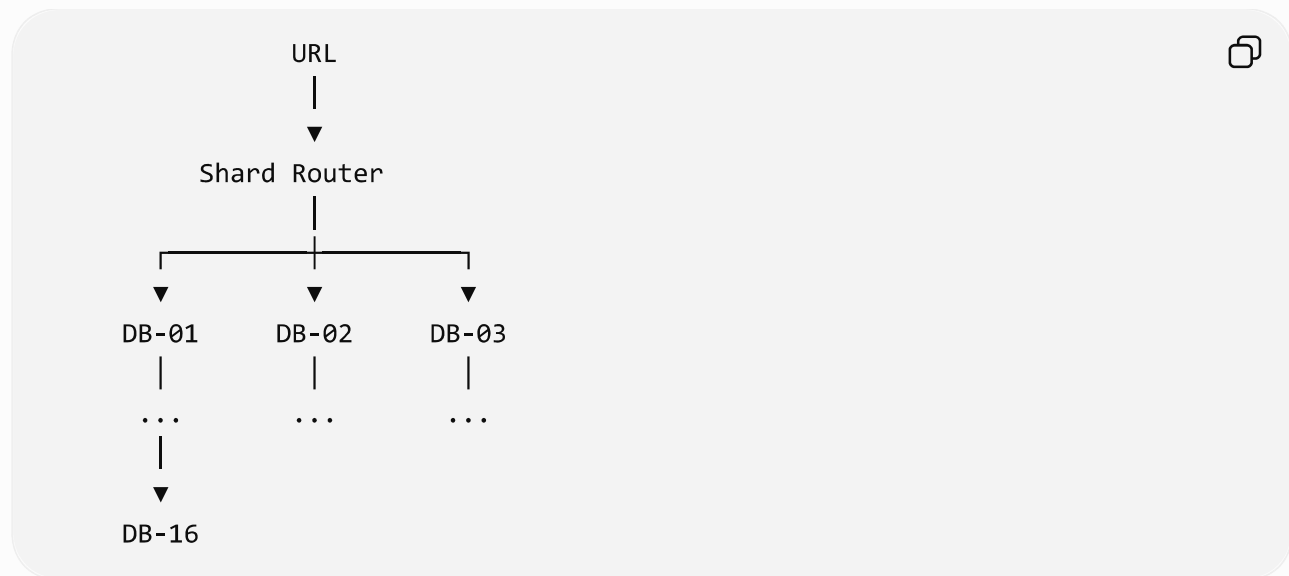


gives:

```
Shard 0  
Shard 1  
Shard 2  
...  
Shard 15
```



Architecture:



Better approach at very large scale

Use **consistent hashing** or a deterministic shard function.

```
shortCode  
|  
▼  
hash()  
|  
▼  
partition  
|  
▼
```



database shard

The application should not scan all databases.

6. How does Redis caching work?

Redirects are overwhelmingly read-heavy.

Imagine:

1 URL creation
↓
10,000 redirects

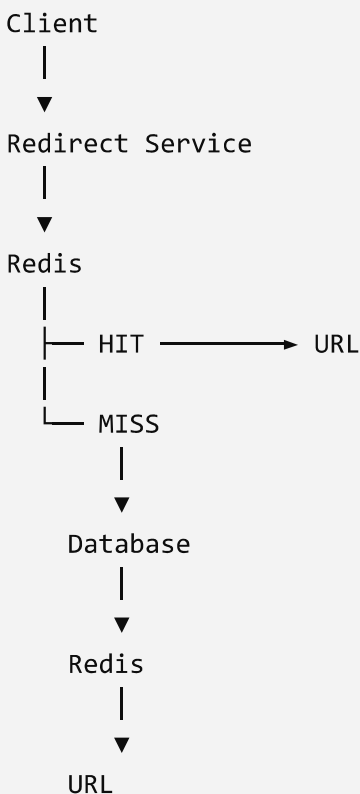


So we don't want:

Every redirect
↓
Database



Instead:



Example:


```
GET /aB92xK
```



Redis:

```
"aB92xK" → "https://example.com/very/long/url"
```

Then:

```
HTTP 301/302
```



```
Location: https://example.com/very/long/url
```

Cache strategy

Use:

```
Cache-aside
```



On cache miss:

```
Redis MISS
```

```
↓
```

```
DB lookup
```

```
↓
```

```
Redis SET
```

```
↓
```

```
redirect
```



A TTL can be used:

```
TTL = 1 hour
```



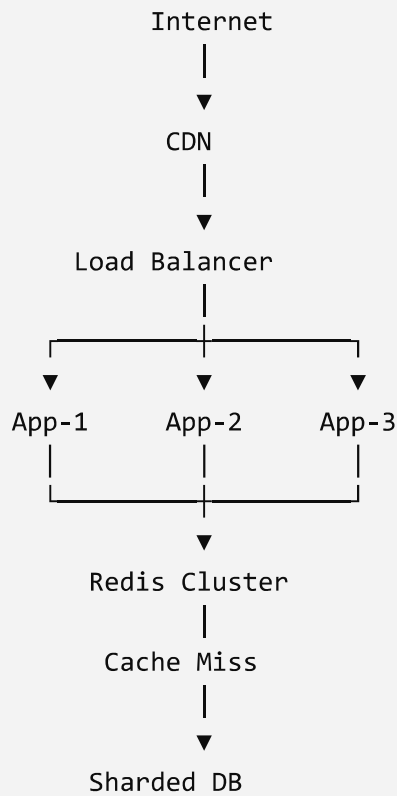
or longer depending on deletion/update requirements.

7. How do we handle millions/billions of redirects?

The critical principle:

Redirect traffic should not hit the primary database for every request.

Use multiple layers:



For extremely popular URLs:

1 short URL
↓
10 million requests

CDN/edge caching can potentially absorb a huge portion of that traffic.

8. How do we collect analytics without slowing redirects?

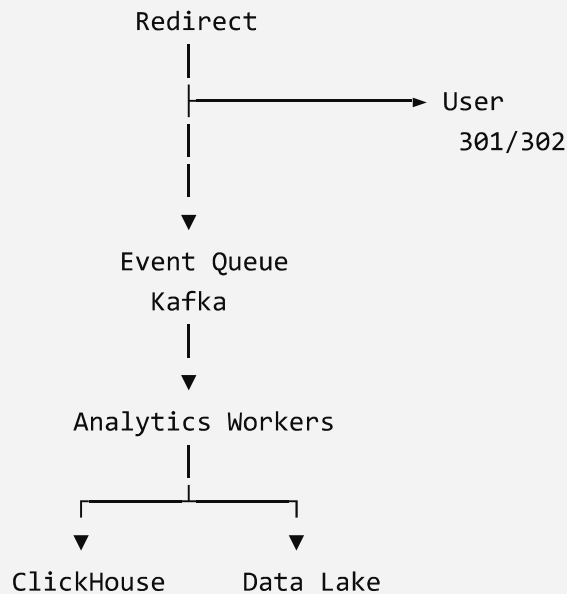
This is extremely important.

Never do this synchronously:

Redirect request
↓
Update analytics DB
↓
Redirect

That adds latency.

Instead:



The redirect service only publishes:

</> JSON

```
{
  "shortCode": "aB92xK",
  "timestamp": "...",
  "country": "IN",
  "userAgent": "...",
  "referrer": "..."
}
```

Kafka buffers the events.

Then consumers process them asynchronously.

This gives:

Redirect latency



very low

Analytics processing



asynchronous

9. How do we handle expired/deleted URLs?

Store metadata with the URL:

URL

- |— short_code
- |— long_url
- |— created_at
- |— expires_at
- |— status
- |— owner_id



For example:

```
short_code = aB92xK
expires_at = 2026-09-01
status = ACTIVE
```



Redirect flow:

GET /aB92xK



Redis



Check expiration



ACTIVE

EXPIRED



301/302

404/410



Important: cache invalidation

If a URL is deleted:

Database



DELETE/UPDATE



Redis DEL



For distributed systems, publish an invalidation event:

URL deleted



Kafka



Redis invalidation



All regions remove cache



For expiration at massive scale, don't create a timer for every URL. Use:

- TTL/indexes where supported
- periodic cleanup workers
- partitioned expiration scans
- lazy expiration on read

10. How do we make it highly available across regions?

Now we're getting into **global architecture**.

Suppose we have:

India

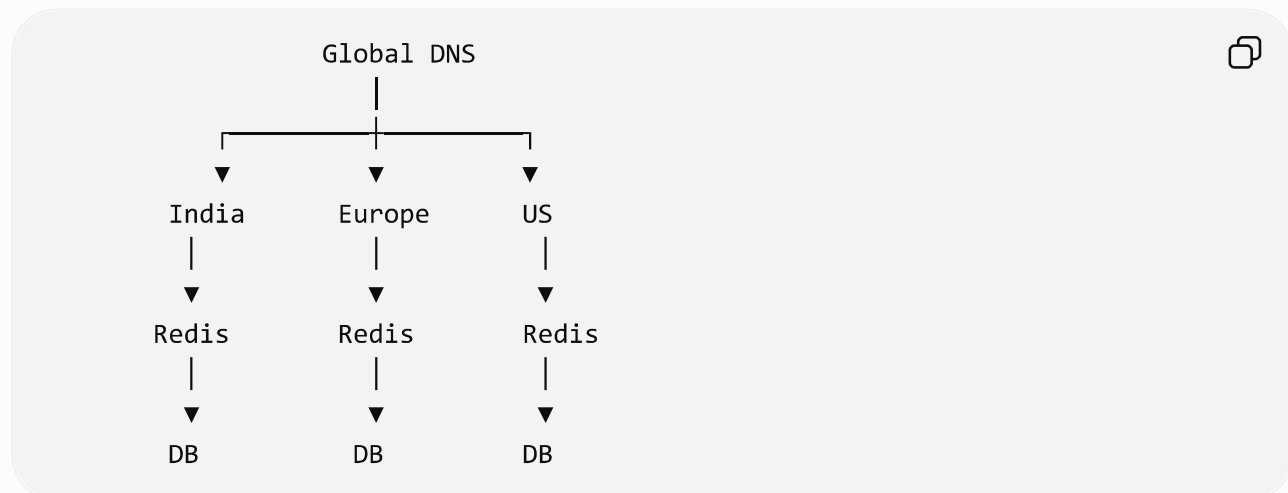
Europe

US

Singapore

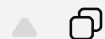


We want users to reach the nearest region.



Use:

Global Load Balancer



+
Geo-routing
+
Regional application clusters
+
Regional Redis
+
Replicated database

Read path

For a redirect:

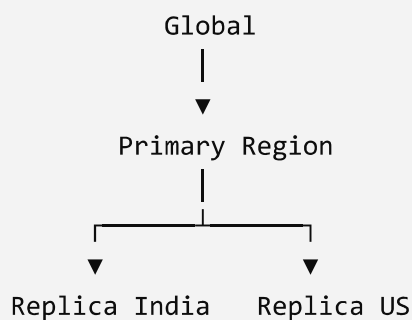
User
↓
Nearest region
↓
Redis
↓
DB if needed
↓
301/302

This keeps latency low.

Multi-region data strategy

There are two major choices.

Option A — Single-writer



Advantages:

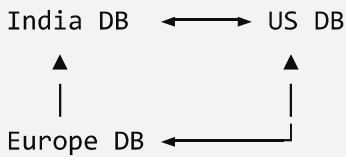
- simpler consistency
- easier ID management

- fewer conflicts

Disadvantage:

- writes depend on primary region

Option B — Multi-writer



Every region can accept writes.

But now you need:

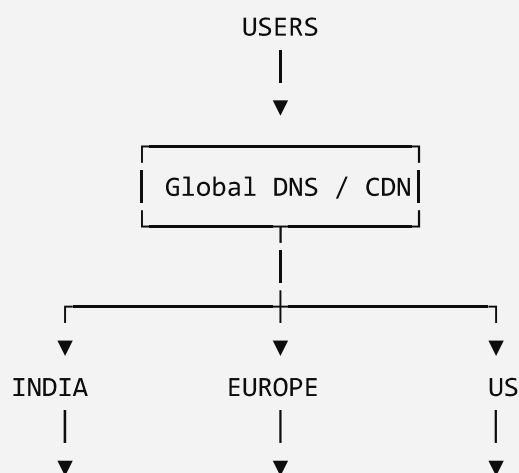
- globally unique IDs
- conflict resolution
- replication
- eventual consistency
- careful deletion handling

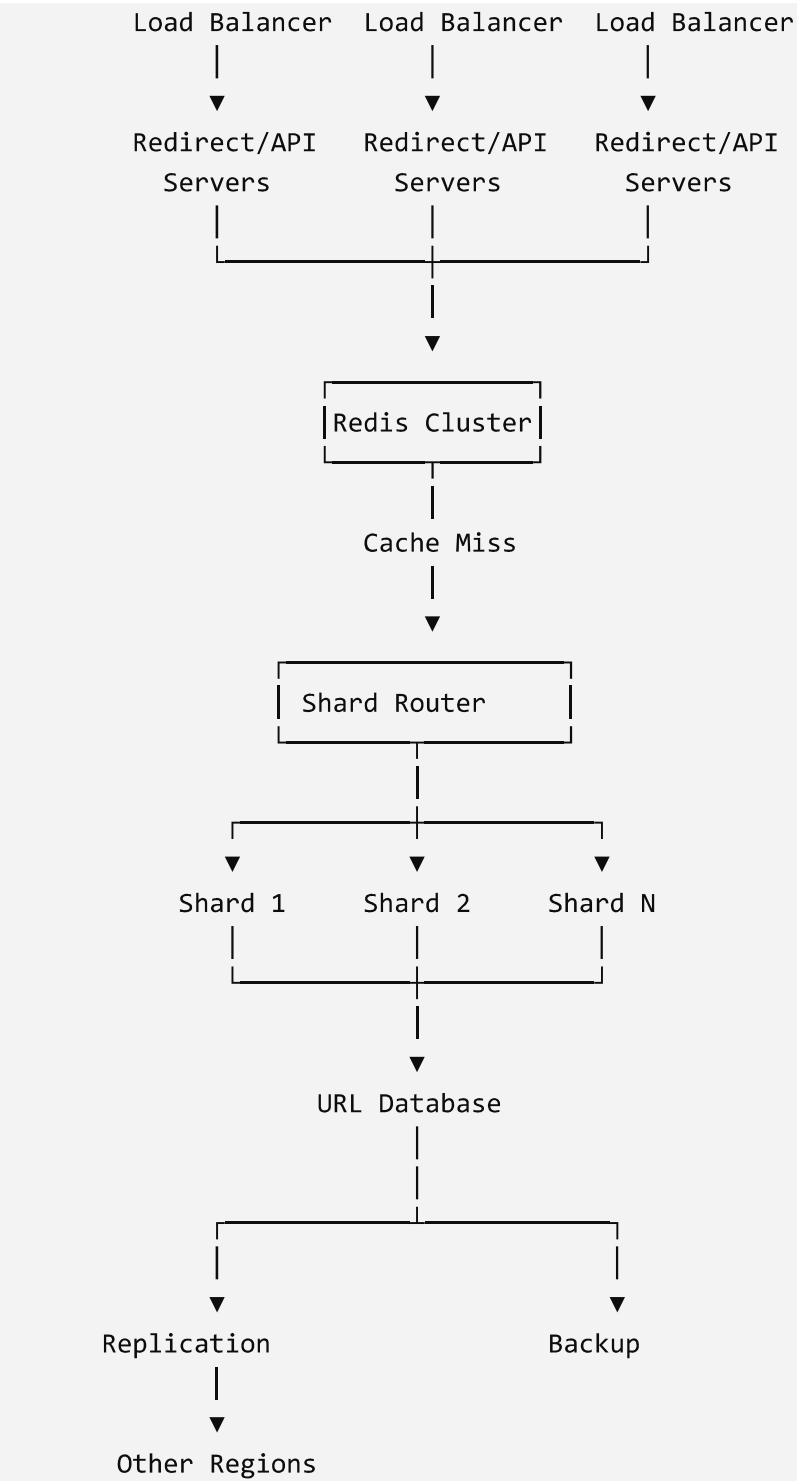
This is significantly more complex.

For a URL shortener, **single-writer + globally replicated reads** is often a very sensible first production architecture.

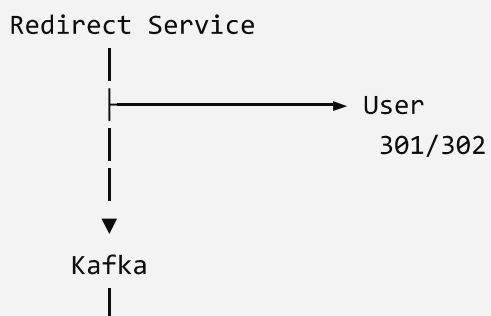
Putting everything together

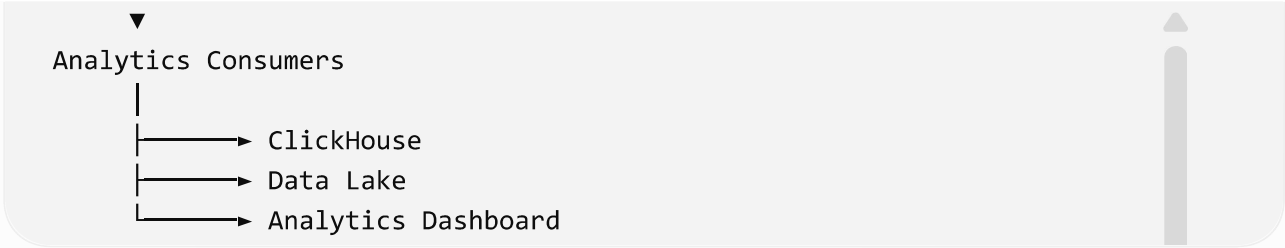
The complete system can look like this:





Redirect Analytics:





The most important design decisions

If I were designing this for a **billions-of-URLs production system**, I'd choose:

Problem	Solution
Unique IDs	Snowflake-style distributed ID
Short representation	Base62
Collision avoidance	Globally unique numeric IDs
Primary storage	Sharded SQL/NoSQL database
Redirect lookup	Redis
Hot URLs	CDN + Redis
Traffic distribution	Global Load Balancer
Analytics	Kafka
Analytics DB	ClickHouse / OLAP store
Expiration	<code>expires_at</code> + lazy/async cleanup
Deletion	DB update + cache invalidation
Multi-region	Regional read replicas/caches
ID generation	Region/node-aware
Availability	Multi-AZ + multi-region
Database scaling	Horizontal sharding
Failure handling	Retries + circuit breakers + replicas

The key interview insight

Don't say:

"I'll use Base62 to generate unique IDs."

Instead say:

"I first generate a globally unique numeric ID using a distributed ID generator such as Snowflake, and then encode that ID using Base62 to obtain a compact URL-safe identifier."

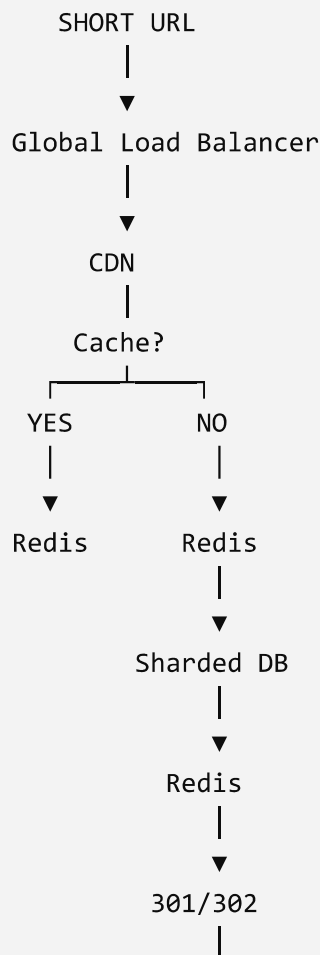
And don't say:

"We'll update analytics before redirecting."

Say:

"The redirect path is latency-critical, so analytics are emitted asynchronously to Kafka and processed independently."

And the overall read path should be:





USER

The diagram shows a system architecture for a URL shortener. At the top, a grey rounded rectangle contains the text 'USER' with a downward arrow above it. Below 'USER' is the flow 'Analytics → Kafka → OLAP'. To the right of this flow is a vertical grey bar with an upward arrow at the top.

Analytics → Kafka → OLAP

That separation—fast synchronous redirect path vs. asynchronous analytics path—is one of the most important architectural ideas in the entire URL-shortener design.



ChatGPT can make mistakes. Check important info.